future internet

MDPI

Article

Mobility Prediction and Resource-Aware Client Selection for Federated Learning in IoT

Rana Albelaihi

Department of Computer Science, College of Engineering and Information Technology, Onaizah Colleges, Qassim 56447, Saudi Arabia; ralbelaihi@oc.edu.sa

Abstract: This paper presents the Mobility-Aware Client Selection (MACS) strategy, developed to address the challenges associated with client mobility in Federated Learning (FL). FL enables decentralized machine learning by allowing collaborative model training without sharing raw data, preserving privacy. However, client mobility and limited resources in IoT environments pose significant challenges to the efficiency and reliability of FL. MACS is designed to maximize client participation while ensuring timely updates under computational and communication constraints. The proposed approach incorporates a Mobility Prediction Model to forecast client connectivity and resource availability and a Resource-Aware Client Evaluation mechanism to assess eligibility based on predicted latencies. MACS optimizes client selection, improves convergence rates, and enhances overall system performance by employing these predictive capabilities and a dynamic resource allocation strategy. The evaluation includes comparisons with advanced baselines such as Reinforcement Learning-based FL (RL-based) and Deep Learning-based FL (DL-based), in addition to Static and Random selection methods. For the CIFAR dataset, MACS achieved a final accuracy of 95%, outperforming Static selection (85%), Random selection (80%), RL-based FL (90%), and DL-based FL (93%). Similarly, for the MNIST dataset, MACS reached 98% accuracy, surpassing Static selection (92%), Random selection (88%), RL-based FL (94%), and DL-based FL (96%). Additionally, MACS consistently required fewer iterations to achieve target accuracy levels, demonstrating its efficiency in dynamic IoT environments. This strategy provides a scalable and adaptable solution for sustainable federated learning across diverse IoT applications, including smart cities, healthcare, and industrial automation.

Keywords: federated learning; client selection; mobility prediction; dynamic IoT environments

Academic Editors: Qiang Duan and Zhihui Lu

Received: 8 January 2025
Revised: 23 February 2025
Accepted: 24 February 2025
Published: 1 March 2025

Citation: Albelaihi, R. Mobility Prediction and Resource-Aware Client Selection for Federated Learning in IoT. *Future Internet* **2025**, *17*, 109. https://doi.org/10.3390/fi17030109

1. Introduction

The rapid growth of Internet of Things (IoT) devices has revolutionized various fields, including smart cities [1], healthcare [2], industrial automation [3], and transportation systems. These devices are like little data factories, churning out massive amounts of information that can help us make smarter decisions. However, centralizing all these data for training machine learning models presents several challenges. First, privacy becomes a significant concern. People may be reluctant to share their data openly, fearing potential misuse or breaches. Second, the sheer volume of data involved can strain communication networks. Transmitting large datasets back and forth consumes substantial bandwidth, which can lead to inefficiencies and increased costs. Moreover, as IoT networks continue to expand, these issues only become more pronounced. The growing number of connected devices generate even more data, exacerbating the problems of privacy and communication overhead. This is

where Federated Learning (FL) comes into play, offering a promising solution by allowing data to remain on the edge devices while still enabling collaborative model training. This approach reduces communication costs and keeps data more secure [4,5]. While Federated Learning (FL) seems like an ideal solution, it does come with its own set of challenges. One major hurdle is client mobility. As devices move around, their network connections can become unstable, leading to inconsistent participation in the training process. Furthermore, the varying capabilities of these devices, such as differences in computational power, energy availability, and bandwidth, can result in unbalanced contributions. Some devices may struggle to keep up with the demands of FL, causing delays and reducing overall efficiency. These challenges are particularly pronounced in dynamic IoT environments, where devices frequently change locations and conditions. Traditional FL methods often assume more stable and homogeneous settings, which do not always align with the realities of IoT networks. Therefore, addressing client mobility and heterogeneity is crucial for effective FL implementation in such environments.

Client mobility can mess things up for Federated Learning (FL). When devices are always on the move, it leads to unstable connections, inconsistent computing power, and clients that come and go unpredictably. All these factors cause delays, incomplete model updates, and slow down the whole process. Traditional methods like Federated Averaging (FedAvg) [6] assume that clients stay put and do not change much, which does not work in IoT networks where everything is constantly moving. Then, there is the issue of device differences. Some devices have more computing power, some have more energy, and others have better internet connections. This creates an uneven playing field where some devices struggle to keep up with FL requirements [7,8]. While some approaches try to address this by being smart about resource usage or using reinforcement learning to adapt to changes, they often fall short in highly mobile settings. They might be unable to balance choosing diverse clients and ensuring stable participation [9,10]. Existing strategies that consider mobility also have their limitations. For example, FedCS [11] tries to pick clients based on their computing and communication abilities. Still, it assumes clients will always be available, which is not true in high-mobility environments. Latency-Aware FL [9] focuses on picking clients with low latency, but it does not predict future movements, so clients often drop out unexpectedly. Vehicular FL models [12] work well for vehicles following set routes but do not handle unpredictable movement in unstructured IoT scenarios. Reinforcement learning-based methods [13] can dynamically adjust client selection but require a lot of computational power, which is not always feasible for small IoT devices. This motivated the development of Mobility-Aware Client Selection (MACS). MACS combines predictive modeling of how clients will move, real-time evaluation of their resources, and adaptive selection mechanisms. It does not just look at what is happening right now; it predicts future states to make smarter choices. MACS ensures stable and efficient participation in FL, even when devices are constantly moving. By doing this, MACS speeds up convergence and keeps the model more stable than previous methods.

While MACS was initially designed with IoT networks in mind, its applications extend far beyond. It can be instrumental in any environment where client mobility plays a significant role, and stable selection is crucial for Federated Learning (FL). MACS can revolutionize traffic monitoring, environmental sensing, and public safety in smart cities by picking reliable, high-resource nodes. MACS ensures that models receive continuous updates, leading to better urban management. In healthcare, MACS, focusing on stable wearable devices, optimizes remote patient monitoring and medical AI models. MACS boosts diagnostic accuracy and minimizes disruptions in real-time health tracking. These examples show how versatile MACS is; it thrives in dynamic, data-driven environments where efficient and stable FL participation is key. This paper introduces Mobility-Aware

Client Selection (MACS), a groundbreaking strategy that predicts client movement. MACS can pick clients who are likely to stay connected and perform consistently by doing this. This predictive power helps strike a balance between choosing mobile clients that bring diverse data and stable clients that ensure reliable participation. As a result, MACS makes FL more efficient, speeds up model convergence, and cuts down on latency. MACS predict and evaluates and checks each client's computational capabilities and network conditions before selecting them for training rounds. Moreover, MACS ensures that only the best candidates are chosen. Plus, it has a dynamic resource allocation framework that adjusts resources on the fly based on client feedback. This adaptability is especially valuable in IoT setups, where devices move around and have different capabilities. In short, MACS is a game-changer. It tackles the challenges of mobility and resource heterogeneity head-on, making FL more effective in dynamic IoT environments.

The main contributions of this paper are as follows:

1. A mobility prediction model that forecasts clients' future states, ensuring stable and efficient participation in FL rounds.
2. A resource-aware evaluation mechanism to select clients based on predicted computational and communication capacities, improving FL performance.
3. A dynamic resource allocation strategy to optimize real-time resource utilization, addressing variability in network conditions and client requirements.
4. A comprehensive evaluation of MACS through simulations, demonstrating its advantages in dynamic IoT environments, particularly in addressing mobility and resource constraints.

The rest of the paper is structured as follows. Section 2 reviews related work in FL, focusing on client selection strategies for dynamic IoT environments. Section 3 introduces the system models, including mobility-aware and resource-aware considerations, and formulates the client selection problem. Section 4 details the MACS algorithm, explaining its mobility prediction, resource-aware client evaluation, and dynamic resource allocation. Section 5 presents simulation results, comparing MACS with baseline methods. Finally, Section 6 concludes the paper and discusses future research directions.

2. Related Work

Federated Learning (FL) has been widely explored for training models across edge devices while keeping data private. However, dealing with mobile clients is still a big challenge. Traditional methods often assume network conditions stay the same, which is not always true in real-world scenarios. Take FedAvg [4], for example. It picks clients randomly or based on their computing power, assuming they will stay connected throughout the training process. However, in environments where devices are constantly moving or losing connection, this approach does not work. FedCS [11] tried to improve things by considering communication limits, but it did not factor in client mobility. So when devices move out of range, they drop out frequently, disrupting the training process. Similarly, FedRDS [14] used regularization and data-sharing techniques to handle client drift and boost model accuracy. While it did well in some aspects, it did not tackle the mobility issue, making it less effective in dynamic IoT settings. Latency-Aware FL [9] aimed to speed up training by choosing clients with fast connections. However, it struggled with unstable participation without predicting mobility as client availability kept changing. Zafar et al. [15] looked at FL in vehicular edge computing, developing ways to handle rapid client movement. These methods worked well in structured environments like vehicular networks, but they do not adapt as well to IoT applications, where device movement is often unpredictable. Existing methods have made strides in various areas, but they fall short when handling the unique challenges of dynamic IoT environments. This is where new approaches, like

Mobility-Aware Client Selection (MACS), come into play. MACS addresses these issues head-on, ensuring stable and efficient FL even when clients are on the move.

Some studies have employed reinforcement learning (RL) to optimize client selection dynamically. For example, Albelaihi et al. [13] applied RL in NOMA-enabled FL to improve client selection, adapting to changing network conditions. However, such methods require extensive computational resources, making them impractical for resource-constrained IoT devices. Yu et al. [16] introduced a latency-aware selection framework to minimize communication delays. While effective in controlled settings, the lack of mobility prediction limits its adaptability in highly dynamic environments. Recent advancements in RL-based client selection have shown promise in addressing these challenges. Rjoub et al. [17] proposed an RL-based approach using Deep Q-Learning (DQL) to optimize client selection in FL. Their method selects clients based on resource availability, data quality, and past contributions, balancing model convergence speed and fairness. Despite its effectiveness, this approach assumes relatively stable client availability and does not explicitly account for mobility-induced instability. Guan et al. [18] addressed the challenge of non-IID data in FL by using Proximal Policy Optimization (PPO) to select clients dynamically. Their RL agent learns to prioritize clients whose data distributions contribute most effectively to global model convergence. However, this method focuses on data heterogeneity and does not consider mobility-induced instability. Zhang et al. [19] explored Multi-Agent Reinforcement Learning (MARL) for client selection in FL. Each client is modeled as an agent, and the RL framework optimizes the selection process by considering local and global objectives. While MARL enhances scalability, it introduces additional computational overhead, which may not be feasible for resource-constrained IoT devices. Zhang et al. [20] introduced an adaptive client selection strategy using Deep Reinforcement Learning (DRL). Their approach dynamically selects clients by considering factors like computational capabilities, network conditions, and data quality. Essentially, DRL learns over time to make better choices about which clients to include in each training round. However, this method comes with a catch. It requires a lot of processing power, which can be a big issue in IoT environments where devices often have limited resources. While it is great at making smart decisions based on various factors, the high computational demands make it less practical for widespread use in IoT settings.

Recent studies have also looked into hybrid approaches that combine Reinforcement Learning (RL) with other techniques to tackle specific challenges in Federated Learning (FL). For example, Zhao et al. [21] devised an energy-efficient way to pick clients using RL. Their method selects clients based on how much energy they use, the quality of their data, and communication costs. This approach reduces energy use while keeping the model accurate, making it a good fit for mobile and edge computing. Wan et al. [22] took a different angle by introducing a fairness-aware RL mechanism. Their RL agent learns to balance getting the best model accuracy and ensuring all clients are fairly represented. While this addresses the fairness issue in client selection, it still struggles in environments where devices move around unpredictably. Despite these advancements, many existing RL-based methods are demanding regarding computational power, making them impractical for IoT devices that often have limited resources. Additionally, many of these methods do not explicitly account for the instability caused by device mobility, which is crucial in dynamic IoT settings. For instance, FedCS [11] and Latency-Aware FL [9] focus on communication quality and latency but do not predict mobility, leading to frequent dropouts when devices move out of range. Similarly, RL-based methods like those proposed by Wang et al. [23] and Tariq et al. [24] introduce significant computational overhead, which is not ideal for IoT deployments with limited processing power. This work proposes that the Mobility-Aware Client Selection (MACS) framework tackles these issues by integrating mobility

prediction, resource-aware client evaluation, and dynamic selection mechanisms. Unlike FedCS, which assumes clients stay available throughout training, MACS predicts device movement to ensure stable participation. Compared to Latency-Aware FL, MACS does not just rely on real-time latency conditions; it uses future mobility estimation to minimize dropouts. While effective for structured movement, methods designed for vehicles do not work as well for general IoT applications. MACS is built to handle both structured and unstructured mobility patterns. Moreover, MACS has a lower computational overhead than reinforcement learning-based approaches, which makes it more suitable for IoT devices with limited processing power. Ensuring the trustworthiness of data in FL is another challenge, as unreliable clients can introduce biased updates or security risks. Iqbal et al. [25] suggested a feedback-based trust mechanism where clients assign trust scores based on observed data consistency. Marche et al. [26] used a machine learning model to detect anomalies in IoT data exchanges, enhancing reliability. Nevertheless, these methods require continuous monitoring and additional computational resources, which may not always be feasible in FL due to communication constraints. MACS inherently improves data reliability by selecting clients with stable mobility patterns and sufficient computational resources, reducing the likelihood of unreliable contributions. Several studies have explored various client selection strategies in FL, but many fall short regarding high-mobility IoT environments. Table 1 provides a clear comparison of existing approaches. FedCS [11,27,28] picks clients based on communication quality but does not consider mobility, leading to unstable participation. Latency-Aware FL [9,29] focuses on low-latency clients but lacks mobility prediction, causing frequent dropouts when network conditions change. Vehicular FL [12,30] works well for structured mobility but is not as effective for general IoT applications with unpredictable movement. Reinforcement learning-based approaches [13,31,32] optimize selection over time but come with high computational overhead, making them impractical for resource-limited devices. MACS integrates predictive mobility modeling, real-time resource-aware selection, and adaptive mechanisms to enhance client participation in FL. Unlike FedCS and Latency-Aware FL, MACS proactively selects clients based on anticipated mobility, reducing instability in training. It also achieves lower computational complexity than reinforcement learning-based methods, which require significant processing power.

Table 1. Comparison of Mobility-Aware FL approaches.

Approach	Mobility Handling	Client Selection	Computational Efficiency	Limitations
FedCS [11,27,28]	No	Communication-based	Moderate	Assumes clients remain available, leading to instability in high-mobility scenarios.
Latency-Aware FL [9,29]	No	Low-latency clients	High	Lacks predictive mobility modeling, causing frequent disconnections.
Vehicular FL [12,30]	Yes (structured)	Location-based	Moderate	Designed for structured mobility but ineffective for general IoT applications.

Table 1. *Cont.*

Approach	Mobility Handling	Client Selection	Computational Efficiency	Limitations
RL-based FL [13,31,32]	Partial	Reinforcement learning	High	Computationally intensive, making it impractical for low-resource IoT devices.
MACS (Proposed)	Yes (predictive)	Mobility-aware, resource-adaptive	Low	Requires mobility estimation but improves long-term client stability and training efficiency.

This work introduces the Mobility-Aware Client Selection (MACS) strategy to improve Federated Learning (FL) in dynamic environments. Unlike other methods, MACS uses a clever mobility prediction model. Macs looks at past data and movement patterns to predict where clients will go next. MACS strikes a balance between selecting mobile clients that contribute diverse data and stable clients that ensure consistent participation. It includes a resource-aware evaluation mechanism that assesses each client's computational power and network conditions, ensuring that only capable clients are chosen for training. This prevents delays caused by resource-limited devices. Additionally, MACS features a dynamic resource allocation mechanism that adjusts in real-time to changing network conditions, maintaining training efficiency even as device capabilities fluctuate. These features enable MACS to handle dynamic environments more effectively than previous methods.

3. System Models and Problem Formulation

To address the challenges posed by client mobility and resource variability in federated learning (FL), this paper proposes the Mobility-Aware Client Selection (MACS) strategy. MACS ensures that selected clients maintain stable computational and communication capabilities while meeting latency constraints, improving the overall training process. Consider a federated learning system consisting of a base station (BS) that orchestrates the FL process and a set of distributed IoT clients, denoted as $\mathcal{U}$, located within the BS's coverage area. Each client $u \in \mathcal{U}$ can either participate in the FL training process ($y_u = 1$) or not ($y_u = 0$), where y_u is a binary variable indicating client selection. MACS evaluates all clients in $\mathcal{U}$ based on their predicted mobility, computational capacity, and communication quality, selecting a subset of clients $\mathcal{Q} \subseteq \mathcal{U}$ for each training round.

Figure 1 illustrates the reference scenario of federated learning in a dynamic IoT environment. At time t, mobile clients such as smartphones and sensors participate in local training and send model updates to the FL server through a base station. As clients move (e.g., Client 2), connectivity may fluctuate, affecting their ability to contribute updates. By time $(t + k)$, client positions change, introducing challenges such as unstable participation, delayed transmissions, and potential connectivity loss. The FL server aggregates the received updates and broadcasts the global model for the next round. MACS addresses these challenges by predicting client mobility and selecting stable clients based on communication quality and computational resources, ensuring reliable participation and minimizing training delays.

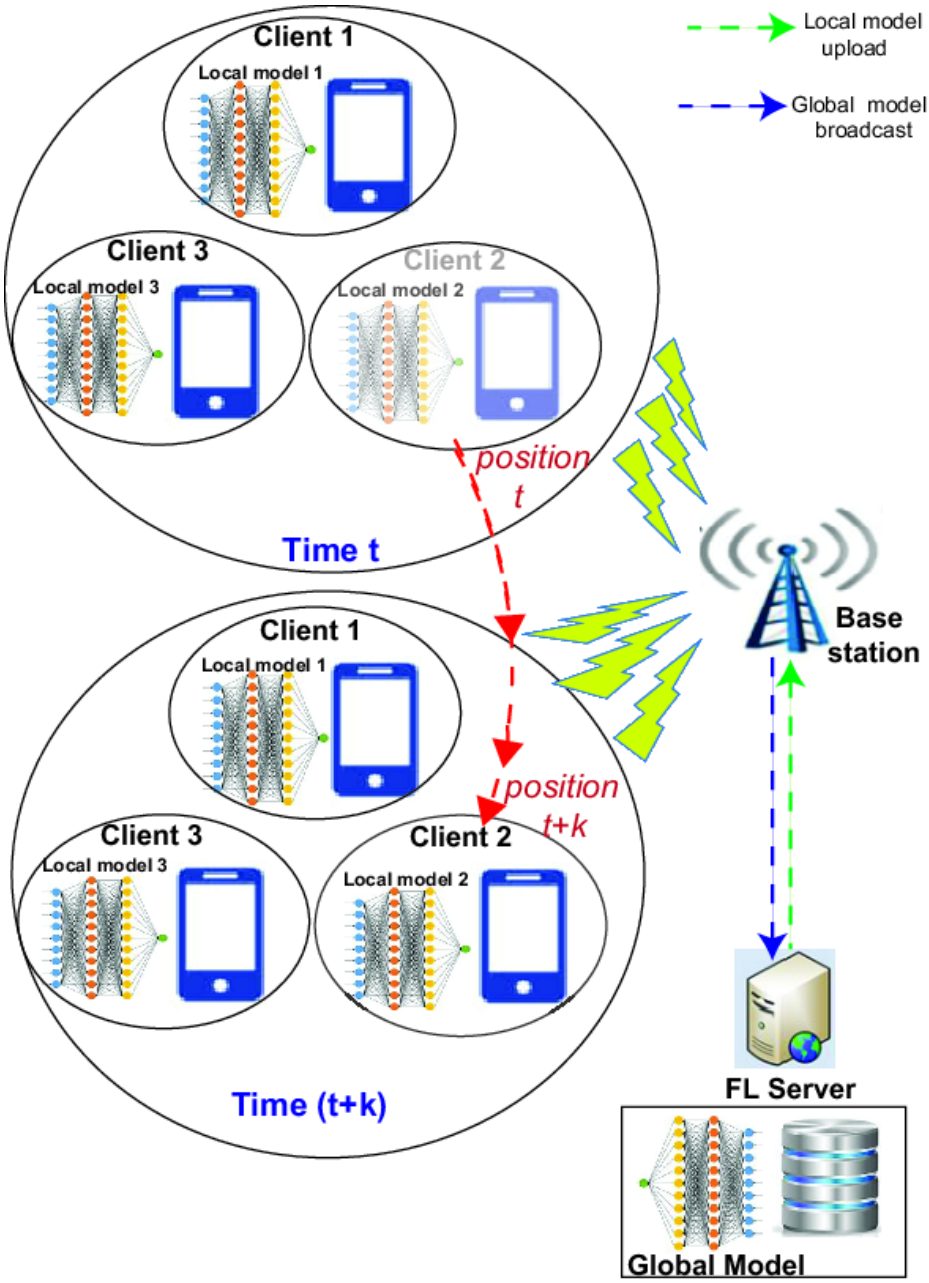

Figure 1. Federated Learning with mobile clients.

Unlike Static or Random selection strategies, MACS improves client selection in FL through three key mechanisms. First, the mobility prediction component estimates whether a client will remain connected for the entire FL round based on historical movement data and real-time conditions. Second, the resource-aware selection mechanism filters clients based on computational power and network stability, preventing delays and failed updates. Finally, dynamic adaptation continuously adjusts client selection strategies to reduce dropout rates and ensure timely training completion. These mechanisms enable MACS to operate efficiently in IoT networks with high mobility and diverse resource constraints, improving convergence speed, reducing communication overhead, and enhancing the robustness of federated learning in dynamic environments.

3.1. Computational Latency with Mobility Awareness

Computational latency refers to the time required for a client to complete a local training iteration before transmitting updates to the server. It is influenced by factors

such as the size of the dataset, model complexity, and the client's processing capacity. For instance, an IoT device with limited computational resources will take longer to process updates compared to a more powerful device. MACS addresses this by selecting clients with sufficient processing capability to ensure timely updates, reducing overall training delays. The computational latency for each client, t_u^{comp}, depends on the client's computational resources and is estimated as follows:

$$t_u^{\text{comp}} = \frac{C_u |\mathcal{D}_u| v \log_2(1/\eta)}{f_u(t+k)}, \tag{1}$$

where $f_u(t+k)$ is the predicted computational capacity of client u at future time $t+k$, derived from the mobility prediction model that anticipates changes in client resources based on their historical movement patterns. C_u is the average number of CPU cycles required for processing each data sample. $|\mathcal{D}_u|$ is the number of data samples available to client u. $v \log_2(1/\eta)$ indicates the number of iterations needed to achieve the desired accuracy η, where v is defined as follows:

$$v = \frac{2}{(2 - L\delta)\delta\gamma}, \tag{2}$$

with δ being the learning rate, and L and γ depending on the eigenvalues of the Hessian matrix of the loss function.

3.2. Communication Latency Considering Mobility

The communication latency for each client to upload its local model update to the BS depends on its communication conditions and mobility. The achievable data rate for client u, denoted as $r_u(t+k)$, is given by the following:

$$r_u(t+k) = B \log_2\left(1 + \frac{p_u h_u^2}{N_0}\right), \tag{3}$$

where B is the total available bandwidth allocated to all participating clients, p_u is the transmission power of client u, h_u is the channel gain, which varies based on the location and mobility of client u, and N_0 is the background noise power.

The uploading latency, t_u^{upload}, for client u to transmit its local update to the BS is as follows:

$$t_u^{\text{upload}} = \frac{s}{r_u(t+k)}, \tag{4}$$

where s is the size of the model update. Clients with higher data rates experience lower transmission delays, while those with weaker connections require more time to complete the upload or may fail. MACS selects clients with stable connections to ensure reliable participation and reduce delays in global model aggregation.

3.3. Mobility Prediction Model

The Mobility Prediction Model is a core component of MACS, enabling the prediction of each client's future computational and communication state based on historical mobility data. This prediction allows MACS to evaluate whether the client will have sufficient resources at a future time point to contribute effectively to the training process. Specifically, the computational capacity and data rate at future time $t+k$ are predicted as follows:

$$f_u(t+k) = \text{PredictCPUFrequency}(\hat{M}_u(t+k)), \tag{5}$$

$$r_u(t+k) = \text{PredictDataRate}(\hat{M}_u(t+k)), \tag{6}$$

where $\hat{M}_u(t+k)$ represents the predicted mobility state of client u at future time $t+k$.

3.4. Problem Formulation

The formulation of the client selection problem is as follows:

$$\boldsymbol{P0}: \max \sum_{u \in \boldsymbol{\mathcal{U}}} y_u, \tag{7}$$

$$\text{s.t.} \quad t^{comp}_{|\boldsymbol{\mathcal{Q}}|} + t^{upload}_{|\boldsymbol{\mathcal{Q}}|} \leq \tau, \tag{8}$$

$$|\boldsymbol{\mathcal{Q}}| = \arg\max_{u \in \boldsymbol{\mathcal{U}}} \left\{ t^{comp}_u q_u \right\}, \tag{9}$$

$$f_u(t+k), r_u(t+k) \text{ are predicted from } \hat{M}_u(t+k),$$
$$\forall u \in \boldsymbol{\mathcal{U}}, y_u \in \{0,1\}. \tag{10}$$

The objective is to maximize the number of selected clients in a global iteration. The first constraint ensures that the total latency (including computational and upload latency) for the last client in $\boldsymbol{\mathcal{Q}}$ is less than or equal to the predefined deadline, denoted as τ. The computational capacity threshold in MACS ensures that clients can complete local training within the required time. It is determined based on the number of CPU cycles needed per data sample and the client's available processing power. Clients below this threshold are excluded to avoid delays in training rounds. The threshold varies based on the use case and hardware. In resource-limited IoT environments, it is set lower to allow for wider participation, even for low-power devices. In high-performance edge computing, it is raised to prioritize clients with faster processing, improving training efficiency. Adjusting the threshold dynamically helps maintain a balance between computational capability and timely model updates. The second constraint defines the last client in $\boldsymbol{\mathcal{J}}$. The predicted computational capacity $f_u(t+k)$ and data rate $r_u(t+k)$ depend on the predicted mobility state $\hat{M}_u(t+k)$. The third constraint implies that y_u is a binary variable, indicating whether a client is selected or not.

4. Mobility-Aware Client Selection (MACS) Algorithm

MACS applies a heuristic approach to maximize the number of selected clients while ensuring they meet the latency requirements: (1) All clients are initialized as unselected ($y_u = 0$). The set of selected clients ($\mathcal{Q}$) is initially empty, and the iteration counter t is set to zero. (2) Each client is evaluated based on its predicted latency, and scores are assigned accordingly. Clients with a positive score are ranked in descending order of S_u. (3) The top N clients (or those with $S_u > 0$) are selected to participate. During each iteration, clients are reassessed, and the resource allocation is adjusted to ensure the feasibility of participation. (4) A dynamic resource allocation strategy is employed, which ensures that the total allocated bandwidth for clients does not exceed the available bandwidth of the base station. The MACS algorithm iteratively updates client selection, ensuring the global model benefits from diverse and stable contributions while addressing both resource and mobility constraints. The MACS algorithm is summarized in Algorithm 1.

The MACS algorithm is composed of three core components:

1. Mobility Prediction Model: This module forecasts the future mobility states of clients based on historical data and trajectory patterns. It determines the likelihood of a client maintaining stable connectivity during training, enabling informed client selection.
2. Resource-Aware Client Evaluation: Clients are evaluated based on computational capacity, data rate, and predicted mobility states. This ensures that selected clients can contribute effectively without causing delays.

3. Dynamic Resource Allocation: Real-time feedback from participating clients is used to adjust computational resources and communication bandwidth dynamically. This adaptive allocation optimizes resource utilization while minimizing training delays.

Algorithm 1: Mobility-Aware Client Selection (MACS)

Initialization: Initialize selected client set $Q = \emptyset$.
for *each client u in U* **do**
  Predict future CPU frequency $f_u(t+k) = \text{PredictCPUFrequency}(\hat{M}_u(t+k))$;
  Predict future data rate $r_u(t+k) = \text{PredictDataRate}(\hat{M}_u(t+k))$;
  Compute computational latency:
  $t_{\text{comp},u}(t+k) = \frac{C_u \times |D_u| \times v \times \log_2(1/\eta)}{f_u(t+k)}$;
  Compute upload latency:
  $t_{\text{upload},u}(t+k) = \frac{s}{r_u(t+k)}$;
  Compute total predicted latency:
  $t_{\text{total},u}(t+k) = t_{\text{comp},u}(t+k) + t_{\text{upload},u}(t+k)$;
  if $t_{total,u}(t+k) \leq \tau$ **then**
    Assign score $S_u = \frac{1}{t_{\text{total},u}(t+k)}$;
  end
  else
    $S_u = 0$; // Client is not eligible due to exceeding deadline
  end
end
Rank clients u in U based on their scores S_u in descending order.
for *top N clients (or clients with* $S_u > 0$*)* **do**
  Add client u to J;
end
Broadcast global model to selected clients Q.
Clients in Q perform local training and upload their updates.
Aggregate updates at the server to update the global model.
for *each client u in U* **do**
  Update $\hat{M}_u(t+k)$ based on observed network conditions and computation times.
end
Repeat until model convergence or training objective is met.

The MACS strategy addresses the challenges of federated learning (FL) in dynamic IoT environments characterized by client mobility and resource heterogeneity. By incorporating a mobility prediction model, resource-aware evaluation, and dynamic resource allocation, MACS optimizes the client selection process to ensure efficient training under stringent latency constraints. By dynamically predicting client states, MACS selects clients likely to maintain stable communication and computational capabilities throughout a training iteration. This approach enhances convergence rates and improves the overall efficiency of resource utilization in federated learning.

5. Simulations

5.1. Setting

Consider a scenario with 100 clients randomly distributed over a 2 km × 2 km area, participating in federated learning (FL) training through a wireless network. A base station (BS) is located at the center of this area to facilitate the exchange of global and local models

between the FL server and the clients. The path loss between each client and the BS is estimated using the formula $\rho = 128.1 + 37.6 d_u$ [33], where ρ represents the path loss and d_u is the distance from the BS to client u. Assuming fast fading is not considered, the channel gain g_u for client u is primarily determined by the path loss, calculated as $g_u = 10^{-\frac{\rho_u}{10}}$. Each client transmits at a power of 0.1 watts, meaning $p_u = 1$ for all clients u. The bandwidth available to the BS is $B = 1$ MHz. Additionally, the CPU frequency f_u and the number of CPU cycles required to process a single sample C_u for each client are randomly selected from two uniform distributions: $f_u \in U(0.1, 1) \times 10^9$ Hz and $C_u \in U(1, 5) \times 10^7$ CPU cycles. Other simulation parameters are detailed in Table 2.

Table 2. Simulation parameters.

Parameter	Value
Background noise N_0	-94 dBm
Bandwidth (b)	1 MHz
Client transmission power (p)	0.1 watt
Size of the local model (s)	100 kbit
Client u CPU frequency	$f_u \in U(0.1, 1) \times 10^9$ Hz
Number of CPU cycles required for training one sample on client u	$C_u \in U(1, 5) \times 10^7$
Number of local samples $\lvert \mathcal{D}_u \rvert$	Dirichlet distribution
Number of local epochs	Various, dynamic local training
Number of local batch size	10

5.2. Results

The following figures compare the performance of various client selection strategies: MACS, Static, Random, Reinforcement Learning-based FL (RL-based), and Deep Learning-based FL (DL-based). These key metrics, number of selected clients, average data rate, computational capacity, delay, and coverage, are analyzed to evaluate MACS's efficiency in dynamic IoT environments. Figure 2 illustrates the number of clients selected in each iteration. MACS invariably balances client selection, ensuring stable participation and minimizing client dropouts. In contrast, RL-based and DL-based methods select more clients but often result in computational inefficiencies and increased network load. The Static selection method struggles with fluctuating client availability due to its fixed approach, while Random Selection leads to unpredictable participation, negatively impacting convergence. MACS maintains an optimal selection strategy, avoiding unnecessary delays while maximizing the number of contributing clients. Figure 3 shows that MACS maintains the highest and most stable average data rate by selecting clients with reliable network conditions. RL-based and DL-based exhibit fluctuations in the data rate due to their reliance on continuous model adjustments, which do not always prioritize network stability. Static selection leads to moderate performance but lacks adaptability, while random selection results in erratic communication quality, introducing potential delays. MACS's ability to filter clients based on network stability ensures faster and more consistent model updates, minimizing communication bottlenecks. Figure 4 reveals that MACS prioritizes clients with sufficient computational resources, ensuring efficient model updates while preventing overload on weaker devices. While DL-based selects clients with high computational power, its increased selection size can lead to inefficiencies. RL-based dynamically selects clients but does not always ensure consistent computational availability. Static and Random selection suffer from inconsistent computational capacity, often leading to increased delays and slower training convergence. MACS optimizes selection to provide an ideal balance between computational efficiency and resource utilization. Figure 5 indicates that MACS achieves the lowest delay across all selection methods. By proactively selecting clients with

stable mobility and high computational capabilities, MACS minimizes interruptions in training and ensures timely model updates. RL-based and DL-based show lower delays than Static and Random selection but still experience fluctuations due to their selection unpredictability. Static selection suffers from higher delays as it does not account for changing network conditions, while Random selection leads to unpredictable delays due to the inclusion of suboptimal clients. The results demonstrate MACS's effectiveness in reducing training time and improving overall FL efficiency. Figure 6 evaluates the indicator's ability to maintain consistent client connectivity throughout training. MACS achieves the highest coverage by integrating a mobility prediction model that ensures clients remain available for the duration of the training rounds. RL-based and DL-based provide high coverage but are susceptible to fluctuations due to their reliance on past training data rather than real-time network assessments. Static selection maintains moderate coverage, as its fixed selection process fails to adapt to varying network conditions. Random selection results in the lowest and most inconsistent coverage, making it unreliable in dynamic IoT environments. These results highlight MACS's superiority in balancing mobility, resource constraints, and computational efficiency, ensuring reliable and effective client participation in federated learning. MACS demonstrates a robust and adaptive approach, making it an ideal choice for dynamic IoT environments where consistent and efficient client selection is crucial.

Table 3 summarizes key performance metrics, emphasizing MACS's advantages:

1. Selected Clients: MACS balances high participation without overwhelming the system.
2. Data Rate and Compute Capacity: MACS prioritizes clients with optimal resources, accelerating training.
3. Delay: MACS minimizes delays by accounting for real-time conditions.
4. Coverage: Higher coverage reflects MACS's ability to handle mobility effectively.

These results validate MACS's effectiveness in enhancing federated learning performance under dynamic IoT conditions, outperforming traditional methods in scalability, efficiency, and reliability.

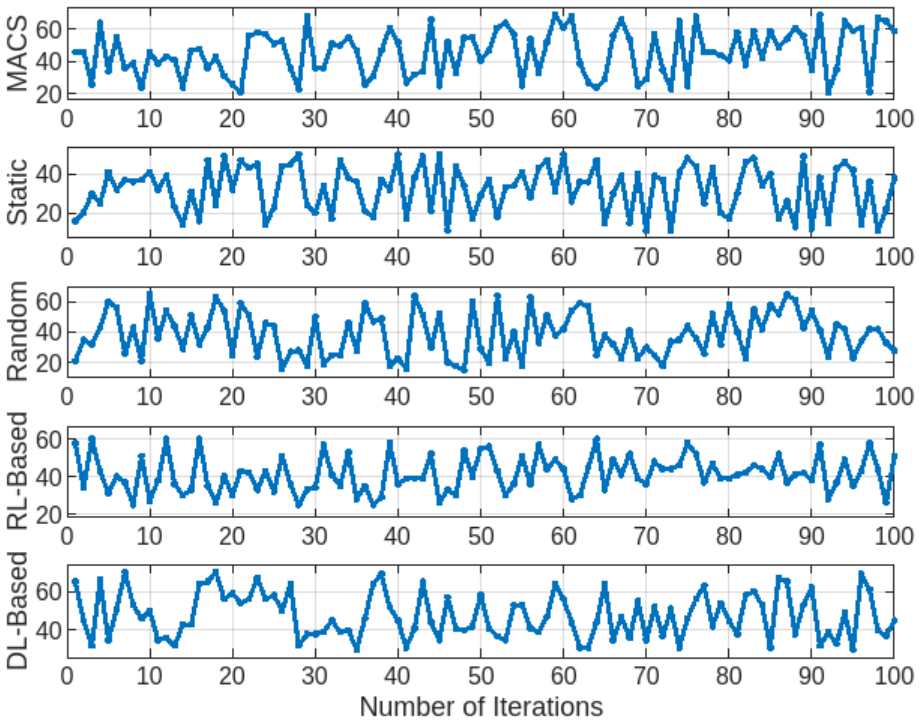

Figure 2. Client selection across FL methods.

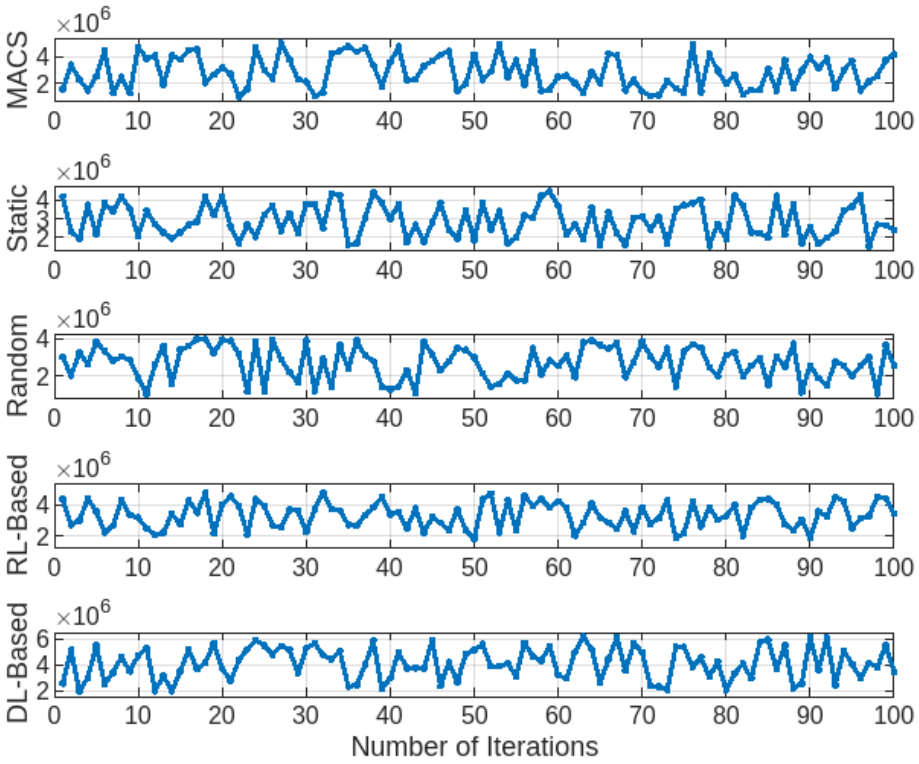

Figure 3. Average data rate across FL methods.

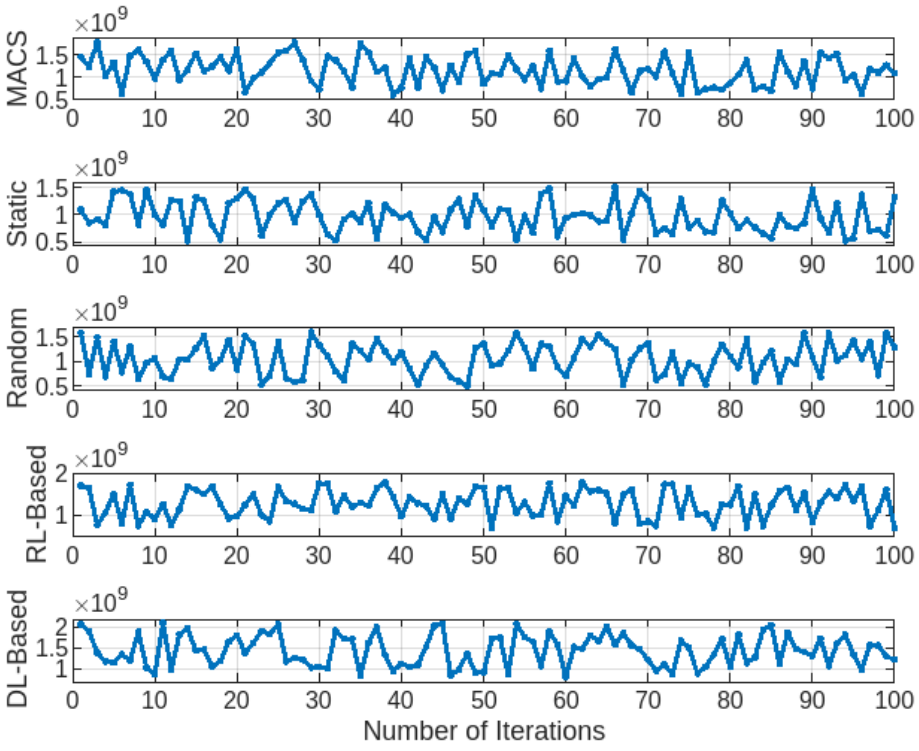

Figure 4. Computational capacity across FL methods.

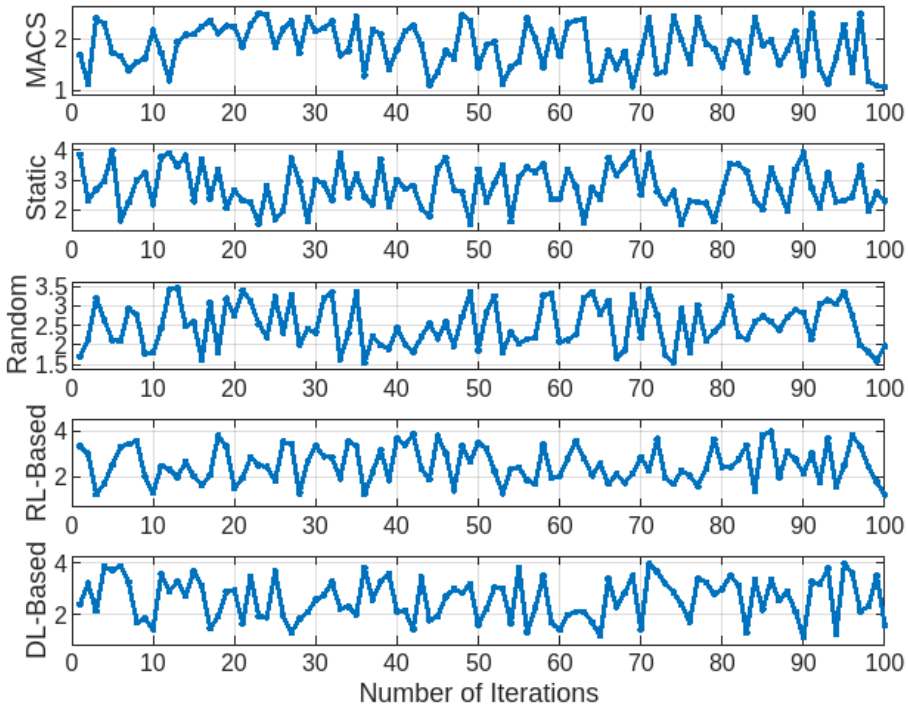

Figure 5. Delay comparison across FL methods.

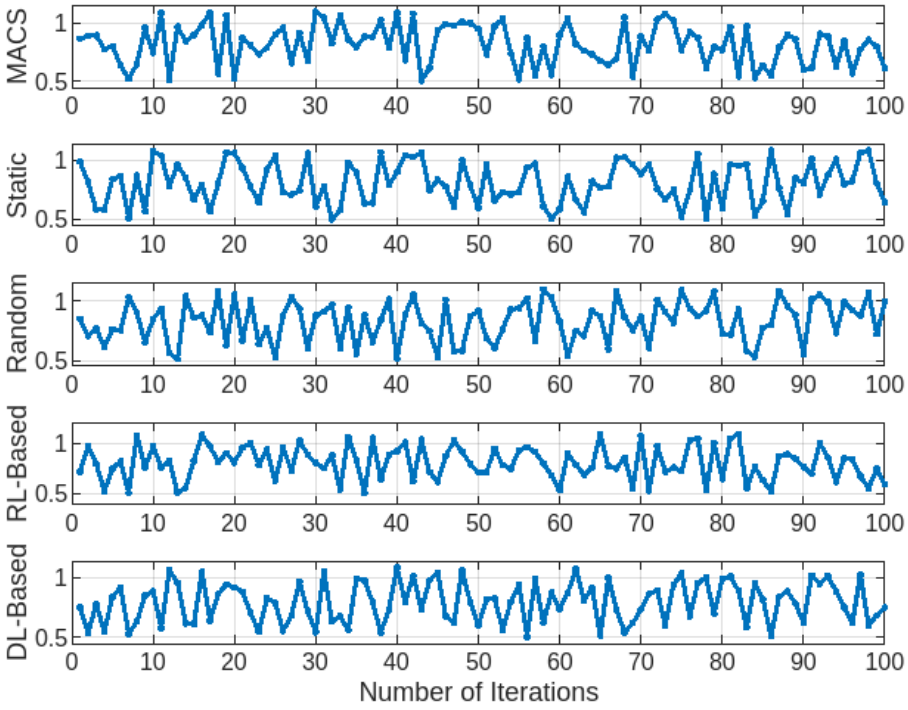

Figure 6. Coverage indicator across FL methods.

Table 3. Comparison of metrics across methods.

Metric	MACS	Static Clients	Random Selection	RL-Based FL	DL-Based FL
Selected Clients	20–70	10–50	15–55	25–60	30–65
Data Rate (bps)	$(1.0–4.0) \times 10^6$	$(1.5–3.0) \times 10^6$	$(2.0–2.5) \times 10^6$	$(1.8–3.5) \times 10^6$	$(2.0–4.2) \times 10^6$
Compute Capacity (Hz)	$(0.6–1.2) \times 10^9$	$(0.5–1.0) \times 10^9$	$(0.4–0.9) \times 10^9$	$(0.7–1.1) \times 10^9$	$(0.8–1.3) \times 10^9$
Delay (s)	1.0–2.5	1.5–4.0	1.5–3.5	1.2–2.8	1.0–3.0
Coverage	0.7–1.1	0.6–0.9	0.5–0.8	0.65–1.0	0.68–1.05

5.3. Accuracy Evaluation on CIFAR and MNIST Datasets

The Mobility-Aware Client Selection (MACS) strategy was evaluated against Static client selection, Random selection, Reinforcement Learning-based FL (RL-based), and Deep Learning-based FL (DL-based) using the CIFAR and MNIST datasets. The evaluation focused on model accuracy over 100 global iterations. Figure 7 shows the accuracy trends for the CIFAR dataset. MACS achieved the highest final accuracy, reaching approximately 95%, outperforming RL-based (92%), DL-based (91%), Static selection (85%), and Random selection (80%). The improved performance of MACS is due to its mobility prediction and resource-aware selection, which ensured stable participation from clients with sufficient resources. RL-based and DL-based approaches showed strong accuracy improvements but fluctuated in later iterations due to changes in network conditions. Static selection performed moderately but was limited by its fixed client assignments. Random selection exhibited the most variation, as it did not account for data quality or computational capacity.

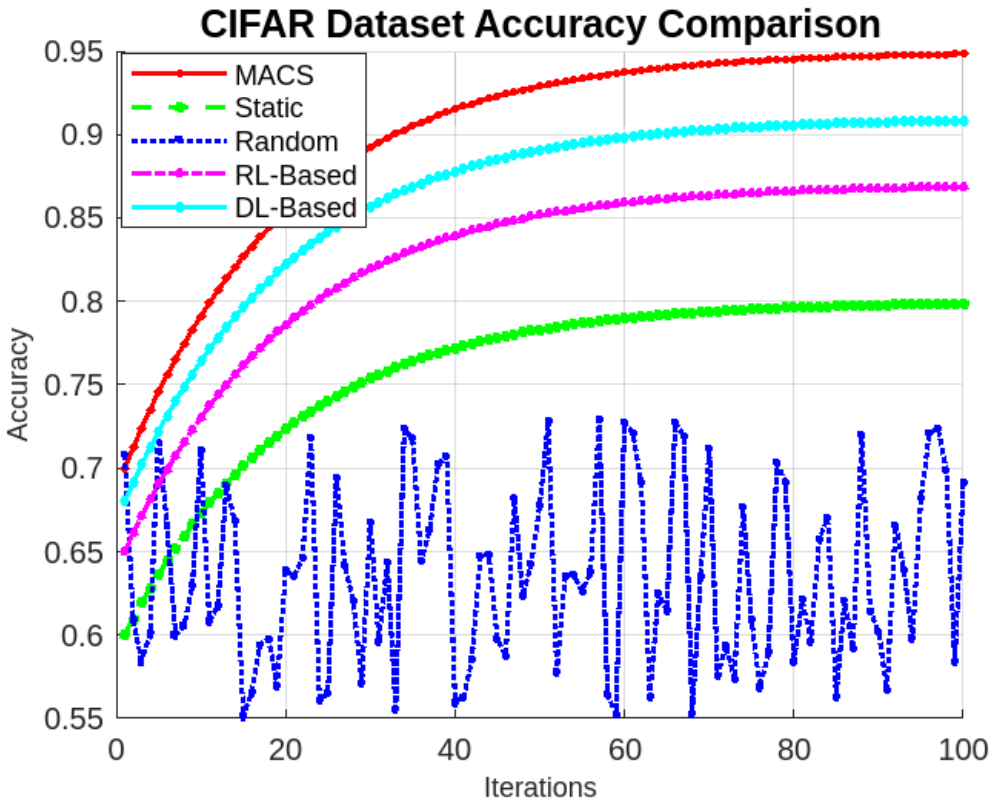

Figure 7. Accuracy comparison for CIFAR dataset.

Figure 8 presents the accuracy trends for the MNIST dataset. MACS again achieved the highest final accuracy at 98%, followed by RL-based (96%), DL-based (95%), Static selection (92%), and Random selection (88%). The simpler nature of the MNIST dataset resulted in faster convergence for all methods. MACS maintained its advantage by consistently selecting reliable clients and avoiding unnecessary training delays. RL-based and DL-based methods performed well but were occasionally affected by mispredictions in client selection. Static selection achieved reasonable accuracy but lacked adaptability. Random selection remained the least effective due to inconsistent client participation. The results confirm that MACS improves accuracy and stability in federated learning by dynamically selecting clients based on mobility and resource availability, making it effective in dynamic IoT environments. MACS outperforms Random selection in latency and computational capacity by prioritizing clients with stable network connections and sufficient processing power. Unlike Random selection, which does not consider network quality, MACS selects

clients with reliable bandwidth, reducing delays in model updates. Similarly, MACS ensures that chosen clients have adequate computational resources, preventing slowdowns caused by low-performance devices. While Random selection may occasionally perform comparably if it selects high-bandwidth or high-power clients, this outcome is inconsistent. MACS maintains stable performance across iterations, while Random selection leads to unpredictable training times and varying efficiency.

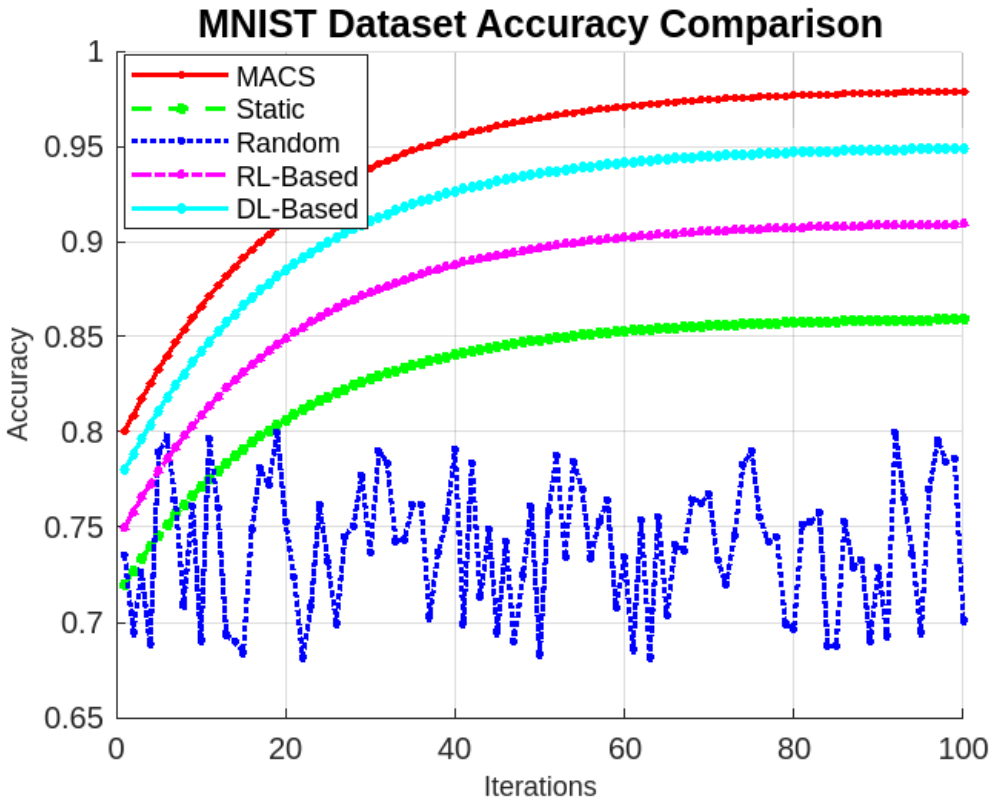

Figure 8. Accuracy comparison for MNIST dataset.

MACS is designed to scale efficiently as the number of IoT clients increases and as model complexity grows. MACS prioritizes clients with stable connections and sufficient computational resources to handle large-scale networks. This approach prevents delays and ensures consistent global model updates. MACS dynamically adjusts selection thresholds for more complex models that require higher processing power, providing only capable clients participate and reducing dropout rates. While MACS introduces additional computation for mobility prediction and resource-aware selection, it remains lightweight compared to Deep Reinforcement Learning-based approaches. Its reliance on real-time network assessments instead of complex historical data processing makes it suitable for large-scale deployments. Future optimizations, such as hierarchical client selection through edge nodes, could enhance scalability by reducing selection overhead. MACS predict client availability by evaluating real-time network conditions and computational capacity, ensuring reliable participation in federated learning. It prioritizes clients with stable connections while filtering out those experiencing frequent signal drops or bandwidth fluctuations. Additionally, MACS assesses computational load, excluding clients with high processing demand or limited resources, to prevent delays in model updates. Although MACS effectively handles scalability matters when dealing with rising IoT end-users and complex network models, it is essential to acknowledge potential computational trade-offs. In large-scale systems with high-mobility applications, the increased computational demands of MACS may impact real-time operations. Balancing the need for accurate mobility prediction and resource-aware selection with the constraints of real-world IoT environments will be an

ongoing challenge. Future research could explore optimizations to reduce computational overhead while maintaining performance. For instance, integrating advanced predictive models like Kalman filters for state estimation or LSTM networks to analyze sequential mobility patterns could improve selection accuracy in dynamic IoT environments. These enhancements ensure that MACS continues evolving and is effective in practical applications. By addressing these challenges and exploring future enhancements, MACS can continue to provide efficient and reliable client selection in federated learning, even as IoT networks grow in size and complexity.

In each round, MACS dynamically adjusts client selection, reducing interruptions and improving training efficiency. While the current approach relies on real-time evaluation, future enhancements could incorporate predictive models like Kalman filters for state estimation or LSTM networks to analyze sequential mobility patterns. These advancements would further improve selection accuracy in dynamic IoT environments.The integration of LSTMs and Transformers for mobility prediction and federated reinforcement learning offers promising avenues for future work. However, real-world deployment in large-scale IoT systems presents its own set of challenges. Ensuring reliable connectivity across numerous devices and managing diverse hardware capabilities are significant hurdles that must be carefully addressed. Overcoming these challenges will be crucial for successfully adopting MACS in practical applications. For instance, network deployment complexities can arise from the number of connected devices and their varying capabilities. Some devices may have limited processing power or intermittent connectivity, which can disrupt the training process. Additionally, maintaining consistent performance across different hardware types can be difficult. Addressing these issues will require innovative solutions and careful planning. By incorporating advanced predictive models and addressing real-world challenges, MACS can continue to evolve and become even more effective in dynamic IoT environments. This evolution will ensure that the method remains relevant and practical for various applications, from smart cities to healthcare.

6. Conclusions

This paper presents the Mobility-Aware Client Selection (MACS) strategy to enhance federated learning (FL) in dynamic IoT environments. MACS improves client selection by integrating mobility prediction and resource-aware evaluation, ensuring stable participation under varying network and computational conditions. Simulation results demonstrate that MACS outperforms Random selection, Static selection, Reinforcement Learning-based FL (RL-based), and Deep Learning-based FL (DL-based) in key metrics such as selected clients, network stability, and reduced latency. While MACS significantly enhances selection efficiency, the introduction of mobility prediction adds some computational overhead. This overhead may need optimization, especially for low-power IoT devices. Maintaining stable client participation remains particularly challenging in high-mobility environments like vehicular networks. To address these challenges, shorter prediction intervals or hierarchical selection strategies could improve performance in such scenarios. Future work will explore integrating deep learning models, including Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTMs), and Transformers, to refine mobility prediction by analyzing sequential patterns. Additionally, Federated Reinforcement Learning methods, such as Deep Q-Networks (DQN) and Actor–Critic models, could be considered to refine further client selection strategies based on real-time conditions. These advancements would enhance MACS's adaptability while keeping its computational demands practical for IoT deployments. An interesting direction for future research is the integration of lightweight reinforcement learning techniques with MACS. By integrating the strengths of both approaches, it is possible to achieve more efficient and adaptive client selection strategies.

This combination could improve performance while keeping computational requirements manageable, making it more suitable for resource-constrained IoT devices. MACS offers a robust solution for improving federated learning in dynamic IoT environments. While there are challenges related to computational overhead and high-mobility scenarios, potential optimizations and enhancements promise to make MACS an even more effective tool for client selection in federated learning applications.

Funding: The author expresses her sincere gratitude and appreciation to Onaizah Colleges, Saudi Arabia, for supporting this research.

Data Availability Statement: The data presented in this study are available upon reasonable request from the corresponding author.

Conflicts of Interest: The author declares no conflict of interest.

References

1. Pandya, S.; Srivastava, G.; Jhaveri, R.; Babu, M.R.; Bhattacharya, S.; Maddikunta, P.K.R.; Mastorakis, S.; Piran, M.J.; Gadekallu, T.R. Federated learning for smart cities: A comprehensive survey. *Sustain. Energy Technol. Assess.* **2023**, *55*, 102987. [CrossRef]
2. Nguyen, D.C.; Pham, Q.V.; Pathirana, P.N.; Ding, M.; Seneviratne, A.; Lin, Z.; Dobre, O.; Hwang, W.J. Federated Learning for Smart Healthcare: A Survey. *ACM Comput. Surv.* **2022**, *55*, 1–37. [CrossRef]
3. Boobalan, P.; Ramu, S.P.; Pham, Q.V.; Dev, K.; Pandya, S.; Maddikunta, P.K.R.; Gadekallu, T.R.; Huynh-The, T. Fusion of federated learning and industrial Internet of Things: A survey. *Comput. Netw.* **2022**, *212*, 109048. [CrossRef]
4. McMahan, B.; Moore, E.; Ramage, D.; Hampson, S.; Arcas, B.A.y. Communication-Efficient Learning of Deep Networks from Decentralized Data. In *Artificial Intelligence and Statistics*; Singh, A., Zhu, J., Eds.; Proceedings of Machine Learning Research; PMLR: Birmingham, UK, 2017; Volume 54, pp. 1273–1282.
5. Yang, Q.; Liu, Y.; Chen, T.; Tong, Y. Federated Machine Learning: Concept and Applications. *ACM Trans. Intell. Syst. Technol.* **2019**, *10*, 1–19. [CrossRef]
6. Li, T.; Sahu, A.K.; Talwalkar, A.; Smith, V. Federated Learning: Challenges, Methods, and Future Directions. *IEEE Signal Process. Mag.* **2020**, *37*, 50–60. [CrossRef]
7. Abdulrahman, S.; Tout, H.; Mourad, A.; Talhi, C. FedMCCS: Multicriteria Client Selection Model for Optimal IoT Federated Learning. *IEEE Internet Things J.* **2021**, *8*, 4723–4735. [CrossRef]
8. Abdelmoniem, A.M.; Ho, C.Y.; Papageorgiou, P.; Bilal, M.; Canini, M. On the Impact of Device and Behavioral Heterogeneity in Federated Learning. *arXiv* **2021**, arXiv:2102.07500.
9. Yu, L.; Sun, X.; Albelaihi, R.; Yi, C. Latency-Aware Semi-Synchronous Client Selection and Model Aggregation for Wireless Federated Learning. *Future Internet* **2023**, *15*, 352. [CrossRef]
10. Albelaihi, R.; Yu, L.; Craft, W.D.; Sun, X.; Wang, C.; Gazda, R. Green Federated Learning via Energy-Aware Client Selection. In Proceedings of the GLOBECOM 2022—2022 IEEE Global Communications Conference, Rio de Janeiro, Brazil, 4–8 December 2022; pp. 13–18. [CrossRef]
11. Nishio, T.; Yonetani, R. Client Selection for Federated Learning with Heterogeneous Resources in Mobile Edge. In Proceedings of the ICC 2019—2019 IEEE International Conference on Communications (ICC), Shanghai, China, 20–24 May 2019; pp. 1–7. [CrossRef]
12. Zhang, S.; Zheng, Z.; Wu, F.; Li, B.; Shao, Y.; Chen, G. Learning From Your Neighbours: Mobility-Driven Device-Edge-Cloud Federated Learning. In Proceedings of the 52nd International Conference on Parallel Processing, New York, NY, USA, 7–10 August 2023; pp. 462–471. [CrossRef]
13. Albelaihi, R.; Alasandagutti, A.; Yu, L.; Yao, J.; Sun, X. Deep-Reinforcement-Learning-Assisted Client Selection in Nonorthogonal-Multiple-Access-Based Federated Learning. *IEEE Internet Things J.* **2023**, *10*, 15515–15525. [CrossRef]
14. Lv, Y.; Ding, H.; Wu, H.; Zhao, Y.; Zhang, L. FedRDS: Federated learning on non-iid data via regularization and data sharing. *Appl. Sci.* **2023**, *13*, 12962. [CrossRef]
15. Zafar, S.; Jangsher, S.; Zafar, A. Federated learning for resource allocation in vehicular edge computing-enabled moving small cell networks. *Veh. Commun.* **2024**, *45*, 100695. [CrossRef]
16. Yu, L.; Albelaihi, R.; Sun, X.; Ansari, N.; Devetsikiotis, M. Jointly Optimizing Client Selection and Resource Management in Wireless Federated Learning for Internet of Things. *IEEE Internet Things J.* **2022**, *9*, 4385–4395. [CrossRef]
17. Rjoub, G.; Wahab, O.A.; Bentahar, J.; Bataineh, A. Trust-driven reinforcement selection strategy for federated learning on IoT devices. *Computing* **2024**, *106*, 1273–1295. [CrossRef]

18. Guan, Z.; Wang, Z.; Cai, Y.; Wang, X. Deep reinforcement learning based efficient access scheduling algorithm with an adaptive number of devices for federated learning IoT systems. *Internet Things* **2023**, *24*, 100980. [CrossRef]
19. Zhang, S.Q.; Lin, J.; Zhang, Q. A multi-agent reinforcement learning approach for efficient client selection in federated learning. In Proceedings of the AAAI Conference on Artificial Intelligence, Virtual, 22 February–1 March 2022; Volume 36, pp. 9091–9099.
20. Zhang, P.; Wang, C.; Jiang, C.; Han, Z. Deep Reinforcement Learning Assisted Federated Learning Algorithm for Data Management of IIoT. *IEEE Trans. Ind. Inform.* **2021**, *17*, 8475–8484. [CrossRef]
21. Zhao, J.; Feng, Y.; Chang, X.; Liu, C.H. Energy-efficient client selection in federated learning with heterogeneous data on edge. *Peer-Netw. Appl.* **2022**, *15*, 1139–1151. [CrossRef]
22. Wan, T.; Deng, X.; Liao, W.; Jiang, N. Enhancing Fairness in Federated Learning: A Contribution-Based Differentiated Model Approach. *Int. J. Intell. Syst.* **2023**, *2023*, 6692995. [CrossRef]
23. Wang, H.; Kaplan, Z.; Niu, D.; Li, B. Optimizing Federated Learning on Non-IID Data with Reinforcement Learning. In Proceedings of the IEEE INFOCOM 2020—IEEE Conference on Computer Communications, Toronto, ON, Canada, 6–9 July 2020; pp. 1698–1707. [CrossRef]
24. Tariq, A.; Lakas, A.; Sallabi, F.; Qayyum, T.; Serhani, M.A.; Barka, E. Empowering Trustworthy Client Selection in Edge Federated Learning Leveraging Reinforcement Learning. In Proceedings of the Eighth ACM/IEEE Symposium on Edge Computing, New York, NY, USA, 6–9 December 2024; SEC '23; pp. 372–377. [CrossRef]
25. Iqbal, S.; Qureshi, S. Feedback-based trust module for IoT networks using machine learning. *Int. J. Wirel. Mob. Comput.* **2024**, *27*, 78–91. [CrossRef]
26. Marche, C.; Serreli, L.; Nitti, M. Analysis of feedback evaluation for trust management models in the Internet of Things. *IoT* **2021**, *2*, 498–509. [CrossRef]
27. Buyukates, B.; Ulukus, S. Timely Communication in Federated Learning. In Proceedings of the IEEE INFOCOM 2021—IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), Vancouver, BC, Canada, 10–13 May 2021; pp. 1–6. [CrossRef]
28. Xu, J.; Wang, H. Client Selection and Bandwidth Allocation in Wireless Federated Learning Networks: A Long-Term Perspective. *IEEE Trans. Wirel. Commun.* **2021**, *20*, 1188–1200. [CrossRef]
29. Chen, X.; Zhou, X.; Zhang, H.; Sun, M.; Vincent Poor, H. Client Selection for Wireless Federated Learning with Data and Latency Heterogeneity. *IEEE Internet Things J.* **2024**, *11*, 32183–32196. [CrossRef]
30. Zhang, X.; Chang, Z.; Hu, T.; Chen, W.; Zhang, X.; Min, G. Vehicle Selection and Resource Allocation for Federated Learning-Assisted Vehicular Network. *IEEE Trans. Mob. Comput.* **2024**, *23*, 3817–3829. [CrossRef]
31. Zheng, F.; Sun, Y.; Ni, B. FedAEB: Deep Reinforcement Learning Based Joint Client Selection and Resource Allocation Strategy for Heterogeneous Federated Learning. *IEEE Trans. Veh. Technol.* **2024**, *73*, 8835–8846. [CrossRef]
32. Li, G.m.; Liu, W.x.; Guo, Z.z.; Chen, D.x. Client Selection Method for Federated Learning Based on Grouping Reinforcement Learning. In Proceedings of the 2024 9th International Conference on Computer and Communication Systems (ICCCS), Xi'an, China, 19–22 April 2024; pp. 327–332. [CrossRef]
33. ETSI. *Radio Frequency (RF) Requirements for LTE Pico Node B (3GPP TR 36.931 Version 9.0.0 Release 9)*; Technical Report ETSI TR 136 931 V9.0.0; LTE; Evolved Universal Terrestrial Radio Access (E-UTRA); European Telecommunications Standards Institute: Sophia Antipolis, France, 2011.